

こどもプログラミング教室 アドバンスコース10-2

キャッスルブレイク 改造後の解答例プログラム

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<script type="text/javascript" src="map.js"></script>
<script type="text/javascript" src="parts.js"></script>
<script type="text/javascript">
```

改造A

改造B

改造C

```
//ゲームの開始の時にやること
function start() {
  for (let i = 0; i < 3; i++) { //【B-1:以下追加】各効果音についてくり返す
    oto[i] = new Audio("oto" + i + ".mp3") //ファイルを開く
    oto[i].preload = "auto" //音が自動で読み込まれるようにする
    oto[i].load() //効果音を読み込んでおく
  }
  timerno = setInterval(mainloop, timer) //タイマーを起動する
}

//キーをおし終えた時にやること
function keyUp(evt) {
  if (mode === 0 && evt.keyCode === 13) { //タイトルモードでEnterキーが押されたら
    oto[0].currentTime = 0 //【B-1:追加】効果音の準備
    oto[0].play() //【B-1:追加】効果音を流す
    setmap(0) //ステージのマップを読み込む
    setbonus() //【C-1:追加】ボーナスブロックをしかける
    animecnt = 0 //【B-2:追加】アニメーションカウンタをセット
    mode = 3 //【B-2:変更】発射待ちアニメモードにする
  } else if (mode === 4 && evt.keyCode === 32) { //発射待ちモードでスペースキーが押されたら
    mode = 5 //プレイモードにする
  } else if (mode === 8 && evt.keyCode === 13) { //ロストモードでEnterキーが押されたら
    ballback() //ボールとパドルの状態を元に戻す
    mode = 4 //発射待ちモードに戻す
  } else if (mode === 7 && evt.keyCode === 13) { //【A-1:以下追加】クリアモードでEnterキーが押されたら
    ballback() //ボールとパドルの状態を元に戻す
    stageno = stageno + 1 //ステージを1つ進める
    if (stageno === wmap.length) { //最終ステージクリア?
      lapno = lapno + 1 //ラップを1つ進める
      stageno = 0 //ステージを先頭に戻す
      if (lapno < 6) { //【C-2:以下追加】ラップが6未満であれば
        wmove = wmove * 2 //城へき移動量を倍にする
      }
    }
    setmap(stageno) //次のステージのマップを読み込む
    setbonus() //【C-1:追加】ボーナスブロックをしかける
    mode = 4 //発射待ちモードに戻す
  }
}

//【C-1:以下追加】ボーナスブロックをしかける
function setbonus() {
  while(true) { //ボーナスの位置が決まるまでくり返す
    let i = Math.floor(Math.random() * cols) //ボーナスの列を乱数で決める
    let j = Math.floor(Math.random() * lines) //ボーナスの行を乱数で決める
    if (nmap[i][j] === 0) { //決まった位置がかべならば
      nmap[i][j] = 3 //ボーナスにする
      return //決定したので戻る
    }
  }
}
```

```
//キーがおされた時にやること
function keyDown(evt) {
  if (mode === 4 || mode === 5) { //発射待ちモードかプレイモードならば
    if (evt.keyCode === 37 && padxy.x > 0) { //←キーが押され左端ではなかったら
      padxy.x = padxy.x - padspeed //パドルを左に動かす
      if (padxy.x < 0) { //端より先に行っていたら
        padxy.x = 0 //端に戻す
      }
    }
    if (mode === 4) { //発射待ちモードならば
      ballxy.x = padxy.x + 36 //ボールも左に動かす
    }
  } else if (evt.keyCode === 39 && padxy.x + pads.x < window1.sx) { //→キーが押され右端ではなかったら
    padxy.x = padxy.x + padspeed //パドルを右に動かす
    if (padxy.x + pads.x > window1.sx) { //端より先に行っていたら
      padxy.x = window1.sx - pads.x //端に戻す
    }
    if (mode === 4) { //発射待ちモードならば
      ballxy.x = padxy.x + 36 //ボールも右に動かす
    }
  }
}
}
```

//一定時間おきにやること

```
function mainloop() {
  if (mode === 5) { //プレイモードならば
    if (walltop.x + wmove < 1 || walltop.x + walls.x * cols + wmove > window1.sx) { //【C-2: 以下追加】城へきが左右端に達したら
      let fixed = false //【C-3: 追加】復活済フラグをオフにしておく
      for (let j = lines - 1; j >= 0; j--) { //【C-3: 以下追加】城へきの全行について逆順に(下から上へ)くり返す
        if (wmove < 0) { //左端なら
          for (let i = 0; i < cols; i++) { //城へきの全列について左から右へくり返す
            if (nmap[i][j] === -1 && (wmap[stageno][i][j] === 0 || wmap[stageno][i][j] === 1)) { //消されたかべか城なら
              nmap[i][j] = wmap[stageno][i][j] //復活させる
              fixed = true //復活済フラグをオン
              break //くり返しを抜ける
            }
          }
        } else { //右端なら
          for (let i = cols - 1; i >= 0; i--) { //城へきの全列について逆順に(右から左へ)くり返す
            if (nmap[i][j] === -1 && (wmap[stageno][i][j] === 0 || wmap[stageno][i][j] === 1)) { //消されたかべか城なら
              nmap[i][j] = wmap[stageno][i][j] //復活させる
              fixed = true //復活済フラグをオン
              break //くり返しを抜ける
            }
          }
        }
      }
      if (fixed) { //復活済フラグがオン?
        break //くり返しを抜ける
      }
    }
    wmove = -wmove //城へき移動の向きを変える
  }
  walltop.x = walltop.x + wmove //【C-2: 追加】城へきをずらす
  if (ballv === 0) { //ボールの方向が左上ならば
    ballxy.x = ballxy.x - ballspeed
    ballxy.y = ballxy.y - ballspeed
  } else if (ballv === 1) { //ボールの方向が右上ならば
    ballxy.x = ballxy.x + ballspeed
    ballxy.y = ballxy.y - ballspeed
  } else if (ballv === 2) { //ボールの方向が右下ならば
    ballxy.x = ballxy.x + ballspeed
    ballxy.y = ballxy.y + ballspeed
  } else if (ballv === 3) { //ボールの方向が左下ならば
    ballxy.x = ballxy.x - ballspeed
    ballxy.y = ballxy.y + ballspeed
  }
  let touch = wallcheck(ballxy) //ボールがかべについたらかべを消し鉄かべならかべにする
  if (touch === 9) { //ボールがついたのが姫なら
    oto[2].currentTime = 0 //【B-1:追加】効果音の準備
    oto[2].play() //【B-1:追加】効果音を流す
    animecnt = 0 //【B-2:追加】アニメーションカウンタをセット
    mode = 6 //【B-2: 変更】姫アニメモードにする
  } else if (touch === 0) { //ボールがついたのがかべか鉄かべなら
    ballv = wallbound(ballv, ballxy) //ボールの向きを変える
  } else { //どちらでもなければ
    if (ballxy.y > window1.sy) { //下端に落ちたら
      if (restball > 0) { //残りボールがあれば
        restball = restball - 1 //残りボール数を減らす
        mode = 8 //ロストモードにする
      } else { //なければ
        oto[1].currentTime = 0 //【B-1:追加】効果音の準備
        oto[1].play() //【B-1:追加】効果音を流す
        mode = 9 //ゲームオーバーモードにする
      }
    } else {
      ballv = padcheck(ballv, ballxy, padxy) //ボールがパドルについたら向きを変える
      ballv = edgecheck(ballv, ballxy) //ボールが上左右端についたら向きを変える
    }
  }
} else if (mode === 3) { //【B-2:以下追加】発射待ちアニメモードならば
  animecnt = animecnt + 1 //アニメーションカウンタをカウントアップ
  if (animecnt > pads.y) { //カウント終了
    mode = 4 //発射待ちモードにする
  }
} else if (mode === 6) { //【B-2:以下追加】姫アニメモードならば
  animecnt = animecnt + 1 //アニメーションカウンタをカウントアップ
  if (animecnt >= window1.sy / 10) { //カウント終了
    mode = 7 //クリアモードにする
  }
}
draw() //画面をえがく
}
```

```
//城へき、ボールなどをえがく
function drawfield() {
  drawwall() //城へきと姫(小)をえがく
  ctx.drawImage(ball[ballv], ballxy.x, ballxy.y) //ボールを表示
  drawrestball(restball) //残りボール数を表示
  drawstageno(stageno) //【A-1:追加】ステージ番号をえがく
}

//画面をえがく
function draw() {
  ctx.drawImage(haikei, 0, 0) //はいけいを表示
  if (mode === 0) { //タイトルモードなら
    for (let j = 0; j < 10; j++) { //行方向に10回くり返す
      for (let i = 0; i < 10; i++) { //列方向に10回くり返す
        if (j === 0) { //最上行であれば
          ctx.drawImage(wall[1], 200 + 24 * i, 72 + 24 * j) //かべ画像[1]を表示
        } else if (i !== 1 && i !== 8 || j !== 1 && j !== 8) { //一つ内側の四すみではなければ
          ctx.drawImage(wall[0], 200 + 24 * i, 72 + 24 * j) //かべ画像[0]を表示
        }
      }
    }
  }
  ctx.drawImage(title, 248, 120) //タイトルを表示
  drawstart() // "Hit ENTER to start"を表示
} else if (mode === 3) { //【B-2:以下追加】発射待ちアニメモードなら
  ctx.drawImage(pad, padxy.x, padxy.y, pads.x, animecnt) //パドルを縮小表示
} else if (mode === 4 || mode === 5) { //発射待ちモードまたはブレイモードなら
  ctx.drawImage(pad, padxy.x, padxy.y) //パドルを表示
  drawfield() //城へき、ボールなどをえがく
} else if (mode === 6) { //【B-2:以下追加】姫アニメモードなら
  ctx.drawImage(himeclear, 192, animecnt * 10 - window1.sy) //姫画像(大・クリア)を表示
} else if (mode === 7) { //クリアモードなら
  ctx.drawImage(himeclear, 192, 0) //姫画像(大・クリア)を表示
  drawnextstage() //【A1:追加】"Hit ENTER to next stage"を表示
} else if (mode === 8) { //ロストモードなら
  drawfield() //城へき、ボールなどをえがく
  drawnext() // "Hit ENTER to next"を表示
} else if (mode === 9) { //ゲームオーバーモードなら
  ctx.drawImage(himelost, 192, 0) //姫画像(大・ロスト)を表示
}
}
</script>

</head>
<body onload="init();" > <!-- 最初に1回だけやること(起動処理)を示す -->
<canvas id="canvas" width="640" height="480" /></canvas> <!-- キャンバスの設置 -->
 <!-- タイトル画像おき場 -->
 <!-- はいけい画像おき場 -->
 <!-- かべ画像[0]おき場 -->
 <!-- かべ画像[1]おき場 -->
 <!-- かべ画像[2]おき場 -->
 <!--【C-1:追加】かべ画像[3]おき場 -->
 <!-- 姫画像(小)おき場 -->
 <!-- パドル画像おき場 -->
 <!-- ボール画像[0]おき場 -->
 <!-- ボール画像[1]おき場 -->
 <!-- ボール画像[2]おき場 -->
 <!-- ボール画像[3]おき場 -->
 <!-- 姫画像(大・ロスト)おき場 -->
 <!-- 姫画像(大・クリア)おき場 -->
</body>
</html>
```